UNO – Reflection on the Two Advanced Topics

Team: Nikolay Valkanov, Danail Slavchev

1. Minimax-Based Computer Opponent

Why we chose it. UNO is played against computer opponents, so how good the bot is decides how much fun the game is. Our first bot (Easy) only plays the first valid card. We wanted a bot that actually plans ahead, so that we can make the game more challenging.

What we learned from research. Minimax looks ahead by assuming each player picks the move that is best for them, and alpha-beta pruning skips branches that cannot change the result. Minimax assumes *perfect information*, though, and in UNO you can't see the other hands or the order of the deck. The usual fix is *determinization* (Perfect Information Monte Carlo): guess the hidden cards many times, run Minimax on each guess, and choose the move with the best average. Card-game programs such as bridge bots have used this approach. A search can't reach the end of a game, so it stops at a fixed depth and scores the position there with an *evaluation function*. That function can be learned from the results of past games instead of being tuned by hand.

How we applied it. Both bots implement a `BotStrategy` interface, so the game can mix Easy and Hard bots with 2–4 players.

- **Guessing the hidden cards.** For every move, `MinimaxBot` builds 24 "possible worlds" by shuffling the cards it hasn't seen and dealing them to the other players and the deck.
- **The search.** In each world it runs Minimax with alpha-beta 5 turns deep. The moves are applied with our real `GameEngine`, so

stacking, skips and reshuffles follow exactly the same rules as the real game.
- **Scoring a position.** At the depth limit the bot estimates its chance of winning from 12 features (hand sizes, wild and action cards, colours held, draw stack, UNO situations) using logistic regression.
- **Learning.** After every game the weights move towards the actual result. They are stored in `resources/bot/minimax-weights.txt`, and a `train` command lets the bot learn by playing against itself.

What we learned from implementing it.

- **Storing games doesn't work.** Our first idea was to save previous games and look them up later. UNO has far too many possible positions for the bot to ever see the same one twice. Learning 12 weights turned out to be the scalable way to "remember" past games.
- **The bot is clearly stronger:**
  - Over 3,000 games against the Easy bot, the Hard bot won 60% without training and about 70% after 20,000 self-play games (two Easy bots score about 50% against each other).
  - Searching deeper than 3–5 turns added almost nothing.
  - A test version that was allowed to see all the cards won 80%, so the remaining gap comes from hidden information and luck.
- **Not every idea helped.** We also tracked which colours an opponent couldn't have (after they had to draw). It worked correctly but made no measurable difference, so we removed it to save computational power.
- **Speed matters.** The bot runs on the Swing event thread, so we kept its slowest move at around 40 ms, quick enough that the window never freezes.

2. Custom Compression for Save Games

Why we chose it. Saving and loading was one of our backlog features.

Instead of just calling `GZIPOutputStream`, we wanted to understand how compression works by writing it ourselves. It fits naturally into our save pipeline: *encapsulation → serialization → compression*.

What we learned from research.

- **Two main families.** Lossless compression comes in two main kinds. *Entropy coders* such as Huffman give frequent symbols shorter codes. *Dictionary coders* such as LZ77/LZSS and LZW replace repeated text with a reference to an earlier copy of it.
- **DEFLATE** (used in ZIP and GZIP) combines both kinds.
- **LZSS** is a simple LZ77 variant. It uses one flag bit per item to tell plain bytes from references, so short matches are never encoded in a way that wastes space.

How we applied it.

1. **Encapsulation and serialization.** The game state is copied into a `SaveState` and serialized as readable `key=value` text (for example `hand=RED:SIX,BLUE:SKIP`).
2. **Compression.** `Compressor` turns that text into LZSS:
    - It searches the previous 4 KB for the longest repeat.
    - A repeat of 3–18 bytes becomes a 2-byte reference: 12 bits of distance and 4 bits of length.
    - One flag byte marks the next 8 items as plain bytes or references.
    - The file starts with a magic number and the original length.
3. **Loading.** Decompression copies the referenced bytes back. It checks every reference, and a corrupt or foreign file produces a `SaveException` instead of a crash.

What we learned from implementing it.

- **Huffman came first.** It only shrank our saves to about 74%, because it saves bits per character and also has to store its code table.
- **LZSS worked much better.** Our data repeats whole words (`YELLOW:`, `DRAW_TWO`), so LZSS shrank saves to about 48%.
- **Combining them didn't help.** Huffman on top of LZSS gave nothing extra, so we kept the simpler single-stage design.
- **Testing with damaged files.** The hardest part was

decompression safety. We tested with many damaged files to make sure loading never crashes the game